

CLAUDE CODE 實戰教材

SmartTrip FX

跟著 Claude Code，從一句需求開始，
做出一支能算旅費、有測試保護的終端機小程式。
不用先會寫程式，複製、貼上、執行、核對答案就好。

AI 負責「哪些行程可能需要現金」這類判斷；
程式 負責加總、比例、門檻與錯誤處理——這條邊界是全書唯一核心。

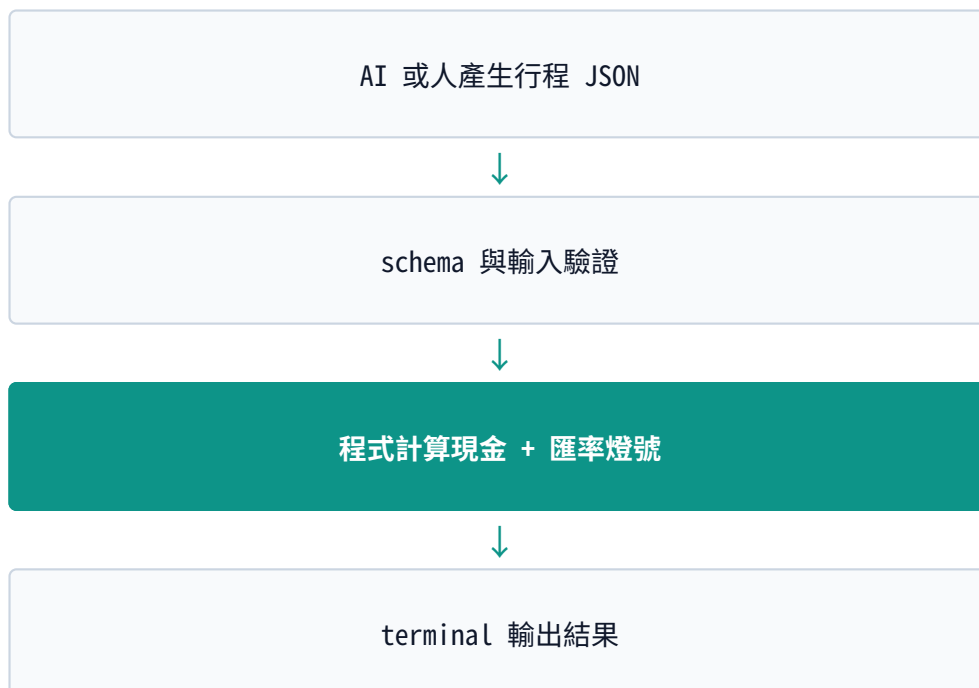
目錄

0	讓 Claude 讀對規則	5
1	建立 project contract（幫 AI 準備一份專案說明書）	7
2	把需求問到沒有歧義（問清楚規則，不留模糊地帶）	9
3	把對話變成可驗收 spec（一份「怎樣算做完」的白紙黑字）	12
4	把 spec 切成三張小票（拆成三個一次做得完的小任務）	14
5	一張票一次完成（實際跑三次「先讓它失敗，再讓它成功」）	16
6	用證據收尾（review 一遍、確認沒問題再存檔）	19
7	把方法帶去下一個專案	21

這是本課第二冊，也是 SmartTrip FX 專案實戰唯一要照著走的文件。請先完成 `CLAUDE-CODE.md` 的官方元件速成，再回到這裡。你不需要先會寫程式，也不用自己決定題目或技術——本書只做同一題、走同一路、用同一組驗收標準。你要做的事很單純：**複製、貼上、執行、跟答案核對**。

看不懂某個英文詞沒關係，這份文件會在第一次出現時，用括號附上白話說明；只要看得懂括號裡的話，就能繼續往下做。

完成品



你可以把它想成兩個角色分工：**AI 是那個很懂行程、幫你猜「這一攤大概要付現金」的朋友**；**程式是六親不認、只認數字的會計**，負責把朋友的猜測換算成明確的金額和燈號。AI 負責「哪些行程可能需要現金」這種判斷；程式負責加總、比例、門檻跟錯誤處理。這條分工邊界，是全書唯一要記住的核心規則。

在開始之前：三個你會一直看到的詞

- **終端機 (Terminal)**：一個只能打字、電腦馬上照做的視窗，跟平常用滑鼠點資料夾不一樣。這本書所有指令都在這裡打。
- **執行一個命令**：把一行文字打進終端機、按 Enter，電腦就會照那行字做事，然後把結果印給你看。

- **JSON**：一種電腦看得懂的清單寫法，用 `{ }` 包資料、用 `,` 分隔欄位。第 5 章會看到實際範例，看過一次就懂。

不用先背起來，走到會用到的地方，這裡都會再提醒一次。

本書怎麼用

每章固定五格：

1. **貼給 Claude**：把整段文字複製起來，貼進你已經打開 `claude` 的那個終端機視窗，按 Enter 送出。
2. **範例問答**：如果 Claude 反問你問題，不用自己想答案，直接把這裡附的建議答案複製貼上就好。
3. **你應看到**：Claude 做完之後畫面大概會長怎樣，重點是「形狀對不對」，不用一字不差。
4. **通過**：貼一段命令去檢查有沒有做對；看到通過的樣子才能往下走，卡住就先別往前。
5. **卡住就貼**：不用自己想怎麼描述問題，直接把這段話貼給 Claude。

本書支援 macOS、Linux 與 Windows WSL。終端機指令都在 repo 根目錄執行。

0 讓 Claude 讀對規則

目標：先確認你電腦上該裝的工具都裝好了，再讓 Claude 弄懂這個專案的規則——哪些只是建議、哪些是系統會自動擋下來的強制規定。

終端機

一行一行貼進終端機、按 Enter：

```
python3 --version
git --version
claude --version
git status --short
```

這四行分別在問電腦：有沒有裝 Python（第幾版）、有沒有裝 Git（幫你保存改動記錄的工具）、有沒有裝 Claude Code、這個資料夾裡有沒有還沒存檔的改動。前三個命令都要回你一行版本號才算成功；如果你是剛下載這個 repo，第四個命令通常什麼都不會印出來，那才是正常的。

貼給 Claude

先讀 CLAUDE.md、.claude/rules/engineering-workflow.md、
.claude/skills/workflow/SKILL.md 與 .claude/settings.json。

現在進入教材第二冊，固定題目是 SmartTrip FX，固定用 Python 3.11+ 的
standard library 寫一支終端機程式（CLI）。先不要寫程式，也先不要 commit（把改動存進 Git
記錄）。

用四行回答我：

1. 這一冊要我完成什麼。
2. 哪些規則只是建議，我可以自己判斷。
3. 哪些操作會被 hook（系統自動檢查）強制擋下來。
4. 我現在唯一該做的下一步是什麼。

你應看到

目標：完成可測試的 SmartTrip FX CLI。
建議：依需求選用 Skills，以 vertical slice 和 TDD 前進。
強制：敏感檔案、credential 與破壞性操作會被 hook 攔截。
下一步：建立 project contract。

(vertical slice 指「一次只做一小片可以獨立驗證的功能」；TDD 指「先寫一個測試證明還沒做到，再寫最少的程式讓測試通過」，第 5 章會實際做一次，現在看不懂沒關係。)

通過

- ☐ Claude 沒有開始寫 code。
- ☐ 它沒有提到 `frame`、`spec`、`evals`、`next` 或 `ship` 這幾個現在已經不存在的舊 Skill 名字。
- ☐ 它給的下一步只有一個，不是一堆選項。

卡住就貼

你引用了目前不存在的流程。請只根據剛讀到的實際檔案重新回答，並用 `rg` 驗證你提到的 Skill 目錄真的存在。

1 建立 project contract (幫 AI 準備一份專案說明書)

目標：幫這個專案寫一份「說明書」，把測試怎麼跑、文件放在哪裡、安全底線都寫清楚。之後不管哪個 Skill 接手，都會先讀這份說明書，不用你每次重新解釋一遍。這份說明書的正式名稱叫 **project contract**。

貼給 Claude

/setup-project

用這門課固定的答案就好，不用讓我自己選技術：

- Runtime: Python 3.11+, 只用 standard library (不裝任何額外套件)。
- Focused test (只跑一小塊測試) : `python3 -m unittest <test_module> -v`
- Full test (跑全部測試) : `python3 -m unittest discover -s tests -v`
- Build check (確認程式能被讀懂、沒有語法錯) : `python3 -m compileall -q smarttrip_fx`
- Typecheck、lint、format : 這門課沒有設定，不要假裝已經驗證過。
- Issue tracker (放任務清單的地方) : 用本機 markdown 檔，放在 `.scratch/smarttrip-fx/issues/`。
- Specs (放規格文件的地方) : `docs/specs/`。
- Git : 用我現在所在的分支；commit message 照 Conventional Commits 格式寫。
- Risk boundary (風險底線) : 刪資料、force push、真的打 API、部署、寫到外部系統，這些動作做之前都要再問我一次。

先去 repo 裡確認一遍，把「你真的從檔案裡看到的事實」跟「這門課固定要求的決定」分開。
先給我一份 `docs/agents/project.md` 的預覽，我確認後你再真的寫進去。先不要動任何產品程式碼。

範例問答

Claude 預覽後，貼：

這份預覽可以用。幫我存成 `docs/agents/project.md`，之後只要跟我說檔案路徑跟你建議的下一步就好。

你應看到

`docs/agents/project.md` 至少要有這幾個區塊：Quality commands（測試怎麼跑）、Issue tracker（任務清單放哪）、Git workflow（怎麼開分支、怎麼寫 commit）、Domain docs（規格文件放哪）、Risk boundary（風險底線）、Verified on（這份說明書是根據什麼、什麼時候驗證過的）。

通過

```
test -f docs/agents/project.md
rg -n "Full test|python3 -m unittest|Risk boundary" docs/agents/project.md
```

兩個命令都必須 exit 0——意思是「這個命令做完了、沒有出錯」。你只要看終端機有沒有跳出紅色錯誤訊息就好，沒有錯誤訊息通常就是過關。

卡住就貼

只修正 `docs/agents/project.md`。缺的指令就寫「未設定」，不要自己去安裝套件，也不要將這門課規定的固定值，寫得好像是你在 repo 裡驗證出來的事實。

2 把需求問到沒有歧義（問清楚規則，不留模糊地帶）

目標：這一章只負責「把規則問到清楚」，先不要碰任何程式碼。「歧義」的意思是：同一句話，兩個工程師可能會理解成不同做法。這一章要把這種模糊全部問掉。

貼給 Claude

```
/grill-with-docs SmartTrip FX
```

請一次只問我一題，確認下面這些已經固定的需求；如果答案已經寫在下面，就不用再問我一次：

產品：SmartTrip FX，幫準備去日本關西玩的人，算大概要換多少日圓現金。

輸入：一個 UTF-8 編碼的 JSON 檔案，裡面有 destination（目的地）、items（行程項目清單）跟 fx（匯率資料）。

items 裡每一筆只有三個欄位：name（名稱）、amount_jpy（日圓金額）、payment（付款方式）。payment 只能是這三種值：cash_only（只能付現）、card_ok（可以刷卡）、unknown（不確定）。fx 裡有 today_twd_per_jpy（今天的匯率）跟 ma30_twd_per_jpy（30 天平均匯率），兩個都用十進位字串表示（例如 "0.2120"）。

換匯規則：

1. 現金小計 = cash_only 的金額加上 unknown 的金額；card_ok 不算進去。
2. 現金小計再加 10% 當預備金，然後無條件進位到下一個 1000 日圓（例如 8030 要進位成 9000）。
3. 今天匯率比 30 天均線低 2% 以上算 GOOD（適合換）；高 2% 以上算 WAIT（先別換）；其他情況算 NEUTRAL（都可以）。
4. JSON 格式錯誤、缺欄位、金額是負的，或 payment 不是上面三種值，程式都要印出清楚的錯誤訊息，而且要用「非 0」的結束代碼（exit code）結束。

這次固定只做：一支只用 Python standard library 寫的終端機程式（CLI）。

不做的部分：真的串 LLM、即時匯率 API、網頁介面、資料庫、登入功能、部署上線。

怎樣算成功：固定的範例輸入要算出 ¥9,000、匯率燈號 GOOD；所有測試都要通過；

全程都不能連網路。

先檢查看看還有沒有「會讓兩個工程師寫出不同做法」的模糊地帶。如果沒有了，

就幫我整理已經決定的事、不做的部分，跟怎樣算過關，然後推薦我下一步要用哪個 Skill。

先不要寫程式。

範例問答

若 Claude 仍追問，依題意貼其中一個答案：

金額規則就用推薦的做法：不確定（unknown）的金額保守算進現金裡，最後的結果無條件進位到 1000 日圓。

匯率門檻就用推薦的做法：差距剛好等於 -2% 算 GOOD，剛好等於 +2% 算 WAIT。

錯誤輸出就用推薦的做法：給人看的錯誤訊息印到 stderr（錯誤輸出）；程式的結束代碼（exit code）用 2。

最後貼：

以上這些決定都對，需求已經問清楚了。請不要再繼續問，直接幫我收斂結論，並推薦下一個 Skill。

你應看到

Claude 應推薦 `/to-spec`，而不是開始實作。它可以先幫忙整理名詞用法（domain glossary），但還不能動手寫產品程式碼。

通過

- ☐ payment 真的只剩三個合法值。
- ☐ 金額公式跟 2% 門檻講法清楚，沒有「大概」「可能」這種模糊字。
- ☐ live API 跟網頁介面（Web UI），清楚寫在不做的範圍裡。
- ☐ 每一條成功條件，都能直接回答「有做到」或「沒做到」。

卡住就貼

現在只找「會讓兩個工程師寫出不同行為」的未知資訊。
若沒有這種未知，停止提問並用已確認需求收斂。

3 把對話變成可驗收 spec（一份「怎樣算做完」的白紙黑字）

目標：讓需求離開聊天紀錄，變成下一次開新對話、別人也看得懂的文件。**Spec** 是一份寫清楚「結果要長怎樣、怎樣算過關」的文件，不是一步一步教你怎麼寫程式的教學。

貼給 Claude

```
/to-spec SmartTrip FX
```

用剛才已經確認的需求，幫我生一份 spec，存到 docs/specs/smarttrip-fx.md。
下面幾個實作上的決定是固定的，不用重新討論：

- package（程式碼放的資料夾）：smarttrip_fx/
- 對外公開的三個函式（seam，也就是外部可以呼叫、也方便單獨測試的介面）：
recommend_cash(items)、fx_signal(today, ma30)、load_trip(path)
- CLI 用法：python3 -m smarttrip_fx examples/kansai-3-days.json
- 只能用 decimal、json、dataclasses、pathlib、argparse、unittest
這些 Python standard library，不裝任何第三方套件。
- 固定範例算出來要是：現金項目 ¥5,500、不確定項目 ¥1,800、
建議換匯 ¥9,000、匯率燈號 GOOD。

每一條驗收條件（acceptance criteria）都要寫成「假設 Given／當 When／那麼 Then」
的格式，而且要能直接回答「過」或「不過」。

先給我看 Problem、Outcome、seams、Out of scope 跟 Open questions 的預覽，
我確認以後你再真的寫進檔案。

範例問答

預覽正確時貼：

確認。Open questions 應為 None，請寫入 docs/specs/smarttrip-fx.md。

你應看到

```
# SmartTrip FX

## Problem
## Outcome
## User stories
## Acceptance criteria
## Implementation decisions
## Testing decisions
## Out of scope
## Open questions
None
```

各區塊白話對照：Problem（要解決的問題）、Outcome（做完之後會有什麼結果）、User stories（誰會怎麼用）、Acceptance criteria（怎樣算過關）、Implementation decisions（技術上的固定決定）、Testing decisions（打算怎麼測）、Out of scope（這次不做的部分）、Open questions（還沒決定的事——這次應該是 None，代表沒有懸而未決的問題）。

通過

```
test -f docs/specs/smarttrip-fx.md
rg -n "Acceptance criteria|recommend_cash|fx_signal|Out of scope|Open questions" docs/specs/smarttrip-fx.md
```

卡住就貼

不要新增新需求。只根據已確認對話補齊可驗收 spec；任何無證據內容放進 Open questions，但課程固定值不得重新變成問題。

4 把 spec 切成三張小票（拆成三個一次做得完的小任務）

目標：每次只讓 Claude 完成一個可驗證的行為，避免一口氣生出整個專案。**Ticket** 只是一份 markdown 檔案，不是要你去哪個平台開真的工單。它的重點是：一次開一個乾淨的新對話，也能獨立做完、獨立驗證。

貼給 Claude

```
/to-tickets docs/specs/smarttrip-fx.md
```

用本機的任務清單（local tracker），固定切成三張票：

1. `.scratch/smarttrip-fx/issues/01-cash-recommendation.md`
做完 `recommend_cash` 跟它的測試。
2. `.scratch/smarttrip-fx/issues/02-fx-signal.md`
做完 `fx_signal` 跟它的測試。
3. `.scratch/smarttrip-fx/issues/03-cli-integration.md`
做完 `load_trip`、CLI、固定的範例檔，以及從頭到尾的整合測試；
這張票要等 01、02 都做完才能開始（blocked by 01、02）。

每張票都要寫：What to build（要做什麼）、Acceptance criteria（怎樣算過關）、Testing seam（要測哪個功能點）、Blocked by（要等哪張票先做完）、Out of scope（不做什麼）。

這門課固定照 01 → 02 → 03 的順序做，不要開 `worktree`，也不要平行做。

先給我看三張票的內容，我確認後你再真的寫進檔案。

範例問答

切分正確。請照剛剛指定的檔名寫入三張 tickets，先不要開始做。

你應看到

```
.scratch/smarttrip-fx/issues/  
├── 01-cash-recommendation.md  
├── 02-fx-signal.md  
└── 03-cli-integration.md
```

通過

```
find .scratch/smarttrip-fx/issues -maxdepth 1 -type f -name '*.md' | sort
```

跑出來的結果一定要剛好三個檔案，而且第 03 張要清楚寫著被 01、02 擋住 (blocked by)。

卡住就貼

不要按 model、service、tests 做水平切分。回到使用者可觀察行為，並嚴格使用教材指定的三個檔名與 dependency。

5 一張票一次完成（實際跑三次「先讓它失敗，再讓它成功」）

目標：實際跑三次 red → green → review，而不是一次生成所有檔案。**TDD** 的意思是：先寫一個一定會失敗的測試，證明「這個功能現在真的還沒做出來」（這叫紅燈 / RED），再寫剛好夠用的程式讓測試通過（這叫綠燈 / GREEN），最後把程式整理乾淨（refactor），但不能改變原本的行為。

5.1 現金計算

貼給 Claude

```
/implement .scratch/smarttrip-fx/issues/01-cash-recommendation.md
```

只做票 01 這一張，先講清楚這次的範圍、怎樣算過關（acceptance criteria）、要測哪個功能點（testing seam），並記住現在這個版本（HEAD）當作基準點。
接著用 unittest 跑一次：寫一個測試 → 讓它先失敗（RED） →
寫最少的程式讓它通過（GREEN） → 再整理程式碼（refactor）。
不要順便做 fx、JSON 解析器或 CLI 這些其他票的事。先不要 commit。

你應看到

至少有 `smarttrip_fx/` 的計算模組與 `tests/test_cash_recommendation.py`，而且回報中包含實際 RED 與 GREEN 命令。

通過

```
python3 -m unittest tests.test_cash_recommendation -v
```


5.2 匯率燈號

貼給 Claude

```
/implement .scratch/smarttrip-fx/issues/02-fx-signal.md
```

只做票 02 這一張。用 Decimal（精準計算用的型別，不會有浮點數誤差）比較 today 跟 ma30，一定要明確測到 -2%、+2% 跟中間值這幾個邊界情況。沿用票 01 已經定好的程式結構（package shape），不要改到現金計算的行為。先不要 commit。

通過

```
python3 -m unittest tests.test_fx_signal -v
```

5.3 JSON 與 CLI 串接

貼給 Claude

```
/implement .scratch/smarttrip-fx/issues/03-cli-integration.md
```

只做票 03 這一張。建立 examples/kansai-3-days.json，內容固定是：

- Airport bus：2300 日圓，cash_only
- Fushimi souvenir：1800 日圓，unknown
- Hotel：24000 日圓，card_ok
- Nishiki market：3200 日圓，cash_only
- today_twd_per_jpy：0.2120
- ma30_twd_per_jpy：0.2180

CLI 跑成功的時候，畫面上一定要有：
目的地 Kansai、現金項目 ¥5,500、不確定項目 ¥1,800、
建議換匯 ¥9,000、匯率燈號 GOOD。

針對「JSON 格式錯誤」「缺欄位」「金額是負的」「payment 不是合法值」這四種情況，都要寫測試檢查程式的對外行為。不要連網路、不要加第三方套件、先不要 commit。

你應看到

```
smarttrip_fx/  
tests/  
examples/kansai-3-days.json
```

檔案可以比這些多，但不能出現 Web framework、database 或 live API client。

通過

```
python3 -m unittest discover -s tests -v  
python3 -m compileall -q smarttrip_fx  
python3 -m smarttrip_fx examples/kansai-3-days.json
```

最後一個命令應包含：

```
目的地: Kansai  
現金項目: ¥5,500  
不確定項目: ¥1,800  
建議換匯: ¥9,000  
匯率燈號: GOOD
```

卡住就貼

先停止加功能。只比對目前 `ticket`、`docs/specs/smarttrip-fx.md` 與失敗測試，
指出一個主要假設、最小可推翻檢查與下一個動作。同一路徑不要嘗試第四次。

6 用證據收尾（review 一遍、確認沒問題再存檔）

目標：review 完整 working tree、修阻擋問題、再產生 commit。Working tree 指你電腦裡目前「還沒存進 Git 記錄」的所有檔案改動。雙軸 review 會分開檢查工程品質（Standards）跟不符合規格（Spec）。

貼給 Claude · 雙軸 review

/code-review HEAD

Spec source（規格來源）：docs/specs/smarttrip-fx.md。

請把 HEAD 之後、整個 working tree 的改動都看過一遍，包含還沒 commit 的（staged）、連 git add 都還沒做的（unstaged），還有全新的檔案（untracked）。

Standards 這一軸檢查正確性、錯誤處理跟測試品質；

Spec 這一軸檢查有沒有漏做、做錯，或做了規格以外的東西（scope creep）。

只列出真的有具體失敗情境的問題（findings），先不要動任何檔案。

若有 blocking finding，貼：

依照 review 給的證據，只修真的會擋路的問題（blocking findings）。

一次修一個，修完馬上重跑最小範圍的測試；全部修完後再跑一次完整測試跟同一個 review。

不要順便擴大範圍去改別的東西。

貼給 Claude · 安全檢查

/security-review

範圍是目前 SmartTrip FX 的 working tree。重點檢查：外部傳進來的 JSON、檔案路徑、錯誤訊息、資源耗用，還有會不會不小心讀到不該讀的敏感檔案。

只回報「有辦法用程式碼證明真的存在」的攻擊路徑，先不要動任何檔案。

通過

```
python3 -m unittest discover -s tests -v
python3 -m compileall -q smarttrip_fx
git diff --check
git status --short
```

tests 必須全綠、compile 與 diff check 必須 exit 0。git status 此時有檔案是正常的，因為還沒 commit。

建立本地 commit

```
git add -A
```

貼給 Claude：

用 commit-message 這個 Skill，根據 staged diff 檢查這次改動夠不夠原子（atomic），給我一個符合 Conventional Commit 格式的 subject。先不要 commit、也不要 push。

若 staged diff 只有本書產物，可直接執行：

```
git commit -m "feat(smarttrip): build deterministic cash recommendation CLI"
git status --short
```

最後一個命令應沒有輸出。到這裡，你已經完成一個有 spec、tickets、tests、review 與可執行入口的專案。

卡住就貼

不要用「看起來沒問題」結案。請列出實際跑過的命令、exit code、未驗證事項，以及目前唯一阻止 commit 的問題。

7 把方法帶去下一個專案

你剛才不是背七個步驟，而是反覆做同一件事：

固定邊界 → 產生一小片 → 用同一個判準驗證 → 人決定是否繼續

需要	使用的 Skill	產出
不知道該走哪條路	<code>/workflow</code>	幫你推薦一條路
需求還有模糊地帶	<code>/grill-with-docs</code>	把決定問清楚
要把對話變成白紙黑字	<code>/to-spec</code>	一份可以拿去驗收的 spec
工作要分好幾次做完	<code>/to-tickets</code>	有先後順序的任務卡
要做完一個功能	<code>/implement</code>	一片測試保護過的功能
準備要交出去	<code>code-review</code> 、 <code>security-review</code>	具體的檢查證據跟問題清單

下一個專案只先貼這一句：

`/workflow` 我想完成 <一句話目標>。先讀 repo 現況，只推薦一條路並說明翻盤條件。

（翻盤條件的意思是：如果之後發現了什麼新狀況，原本推薦的做法就該換掉——先問清楚這個，比盲目照做安全。）

建議先把這整套流程原封不動地再走一次，練熟了以後，再考慮加 live LLM adapter、Web UI 或即時匯率 API 這些真正的複雜度。一次只加一種新東西，不要一次全加。